

Stock Portfolio Tracker

SQL Query and Feature Usage Report

This report explains where every SQL query is located, which feature uses it, which Java method runs it, what parameters are passed into it, and how its result affects the application.

Quick Summary

All runtime SQL queries are inside DAO classes under `src/com/tracker/dao`. The Swing view classes do not write SQL directly; they call DAO methods. `schema.sql` contains setup SQL: database/table creation and demo seed data.

Feature to Query Map

Feature	View method that triggers it	DAO method(s)	Query IDs
Register new account	RegisterFrame.handleRegister()	UserDAO.registerUser()	Q1
Login	LoginFrame.handleLogin()	UserDAO.authenticate()	Q2
Load portfolio sidebar	DashboardFrame.loadPortfolios()	PortfolioDAO.getPortfoliosByUser()	Q3
Create new portfolio	DashboardFrame.createNewPortfolio()	PortfolioDAO.createPortfolio()	Q4
Load holdings and compute dashboard totals	DashboardFrame.loadHoldings()	HoldingDAO.getHoldingsByPortfolio()	Q5
Open trade dialog and populate stocks	TransactionDialog.initComponents()	StockDAO.getAllStocks()	Q10
Autofill latest trade price	TransactionDialog stock selection listener	StockDAO.getLatestClosePrice()	Q11
Execute BUY order	TransactionDialog.executeTrade()	TransactionDAO.executeBuy(), logTransaction(), HoldingDAO.findHolding(), insertHolding()/updateHolding()	Q13, Q6, Q7/Q8
Execute SELL order	TransactionDialog.executeTrade()	TransactionDAO.executeSell(), logTransaction(), HoldingDAO.findHolding(), updateHolding()/deleteHolding()	Q6, Q13, Q8/Q9
Load recent transactions	DashboardFrame.loadTransactions()	TransactionDAO.getRecentTransactions()	Q12
Simulate moving market prices	DashboardFrame.startAutoRefresh()	StockDAO.simulateLatestPriceMovement()	Q14

Database Setup SQL

schema.sql runs before the Java app. It creates the database structure and inserts demo data used by login, dashboard, trades, and market-price simulation.

Location	Section	SQL	Used for
schema.sql lines 6-12	Database cleanup/setup	CREATE DATABASE / USE / DROP TABLE IF EXISTS Alert, Anomaly	Creates/selects the app database and removes old anomaly tables if an older version was initialized.
schema.sql User section	User table	CREATE TABLE IF NOT EXISTS User (...)	Supports registration and login.
schema.sql Portfolio section	Portfolio table	CREATE TABLE IF NOT EXISTS Portfolio (...)	Stores named portfolios belonging to users.
schema.sql Stock section	Stock table	CREATE TABLE IF NOT EXISTS Stock (...)	Stores tradable stock master data.
schema.sql Holding section	Holding table	CREATE TABLE IF NOT EXISTS Holding (...)	Stores current owned stock quantities and average buy prices.
schema.sql Transaction section	Transaction table	CREATE TABLE IF NOT EXISTS Transaction (...)	Stores trade history.
schema.sql Market_Data section	Market_Data table	CREATE TABLE IF NOT EXISTS Market_Data (...)	Stores OHLCV price history. Latest close is used as current price and is updated by the simulator.
schema.sql seed section	Seed inserts	INSERT INTO Stock / Market_Data / User / Portfolio / Holding / Transaction	Adds demo data for immediate testing.

Runtime Query Details

Q1 - Create Account / Registration

Location: src/com/tracker/dao/UserDAO.java:25

Java method: UserDAO.registerUser(name, email, password)

Triggered by: RegisterFrame.handleRegister() at src/com/tracker/view/RegisterFrame.java:177

Query type: INSERT

Tables used: User

SQL:

```
INSERT INTO User (name, email, password_hash) VALUES (?, ?, ?)
```

Parameters: 1=name, 2=email, 3=SHA-256 password hash

Purpose: Stores a new user after UI validation. The raw password is never inserted directly; UserDAO.hashPassword() creates the stored hash.

Result in app: executeUpdate() > 0 means registration succeeded.

Q2 - Login

Location: src/com/tracker/dao/UserDAO.java:53

Java method: UserDAO.authenticate(email, password)

Triggered by: LoginFrame.handleLogin() at src/com/tracker/view/LoginFrame.java:162

Query type: SELECT

Tables used: User

SQL:

```
SELECT * FROM User WHERE email = ? AND password_hash = ?
```

Parameters: 1=email typed by user, 2=SHA-256 hash of typed password

Purpose: Checks whether the submitted credentials match a stored account.

Result in app: If a row exists, UserDAO builds a User object and LoginFrame opens DashboardFrame.

Q3 - Dashboard portfolio sidebar

Location: src/com/tracker/dao/PortfolioDAO.java:24

Java method: PortfolioDAO.getPortfoliosByUser(userId)

Triggered by: DashboardFrame.loadPortfolios() at src/com/tracker/view/DashboardFrame.java:312

Query type: SELECT

Tables used: Portfolio

SQL:

```
SELECT * FROM Portfolio WHERE user_id = ?
```

Parameters: 1=currentUser.getUserId()

Purpose: Loads only the logged-in user's portfolios.

Result in app: Returns Portfolio objects shown in the left sidebar JList.

Q4 - New Portfolio button

Location: src/com/tracker/dao/PortfolioDAO.java:55

Java method: PortfolioDAO.createPortfolio(userId, name)

Triggered by: DashboardFrame.createNewPortfolio() at src/com/tracker/view/DashboardFrame.java:480

Query type: INSERT

Tables used: Portfolio

SQL:

```
INSERT INTO Portfolio (user_id, name) VALUES (?, ?)
```

Parameters: 1=currentUser.getUserId(), 2=name from JOptionPane

Purpose: Creates a new portfolio owned by the current user.

Result in app: executeUpdate() > 0 means the portfolio was created; DashboardFrame reloads the sidebar.

Q5 - Dashboard holdings table and total P/L

Location: src/com/tracker/dao/HoldingDAO.java:25

Java method: HoldingDAO.getHoldingsByPortfolio(portfolioId)

Triggered by: DashboardFrame.loadHoldings() at src/com/tracker/view/DashboardFrame.java:332

Query type: SELECT with JOIN and subquery

Tables used: Holding, Stock, Market_Data

SQL:

```
SELECT h.*, s.ticker, s.company_name,  
(SELECT md.close_price FROM Market_Data md  
WHERE md.stock_id = h.stock_id ORDER BY md.date_id DESC LIMIT 1) AS current_price  
FROM Holding h  
JOIN Stock s ON h.stock_id = s.stock_id  
WHERE h.portfolio_id = ?
```

Parameters: 1=selectedPortfolio.getPortfolioId()

Purpose: Combines portfolio holdings with stock metadata and latest market price in one dashboard query.

Result in app: Returns Holding objects with ticker, company name, and latest close price. DashboardFrame computes total value and profit/loss from these objects.

Q6 - BUY/SELL validation and holding update decision

Location: src/com/tracker/dao/HoldingDAO.java:64

Java method: HoldingDAO.findHolding(portfolioId, stockId)

Triggered by: TransactionDAO.executeBuy() at line 39 and executeSell() at line 63

Query type: SELECT

Tables used: Holding

SQL:

```
SELECT * FROM Holding WHERE portfolio_id = ? AND stock_id = ?
```

Parameters: 1=portfolioId, 2=stockId

Purpose: Detects whether the selected portfolio already owns the stock. BUY uses it to decide insert vs update; SELL uses it to check available quantity.

Result in app: Returns an existing Holding or null.

Q7 - BUY new stock not already held

Location: src/com/tracker/dao/HoldingDAO.java:94

Java method: HoldingDAO.insertHolding(portfolioId, stockId, quantity, avgPrice)

Triggered by: TransactionDAO.executeBuy() at src/com/tracker/dao/TransactionDAO.java:49

Query type: INSERT

Tables used: Holding

SQL:

```
INSERT INTO Holding (portfolio_id, stock_id, quantity, avg_buy_price) VALUES (?, ?, ?, ?)
```

Parameters: 1=portfolioId, 2=stockId, 3=quantity, 4=buy price

Purpose: Adds a stock to the portfolio when the first BUY order for that stock is executed.

Result in app: Creates the initial holding row for that stock.

Q8 - BUY existing stock / SELL partial stock

Location: src/com/tracker/dao/HoldingDAO.java:116

Java method: HoldingDAO.updateHolding(holdingId, newQuantity, newAvgPrice)

Triggered by: TransactionDAO.executeBuy() at line 47 and executeSell() at line 81

Query type: UPDATE

Tables used: Holding

SQL:

```
UPDATE Holding SET quantity = ?, avg_buy_price = ? WHERE holding_id = ?
```

Parameters: 1=newQuantity, 2=newAvgPrice, 3=holdingId

Purpose: For BUY, stores the increased quantity and weighted average buy price. For partial SELL, stores the reduced quantity while keeping average price unchanged.

Result in app: Updates the existing holding row.

Q9 - SELL all shares of a stock

Location: src/com/tracker/dao/HoldingDAO.java:137

Java method: HoldingDAO.deleteHolding(holdingId)

Triggered by: TransactionDAO.executeSell() at src/com/tracker/dao/TransactionDAO.java:78

Query type: DELETE

Tables used: Holding

SQL:

```
DELETE FROM Holding WHERE holding_id = ?
```

Parameters: 1=holdingId

Purpose: Keeps the holdings table clean by removing positions that no longer exist.

Result in app: Removes the holding when remaining quantity becomes zero.

Q10 - Trade dialog stock dropdown

Location: src/com/tracker/dao/StockDAO.java:23

Java method: StockDAO.getAllStocks()

Triggered by: TransactionDialog initComponents() at src/com/tracker/view/TransactionDialog.java:81

Query type: SELECT

Tables used: Stock

SQL:

```
SELECT * FROM Stock ORDER BY ticker
```

Parameters: None

Purpose: Populates the JComboBox so the user can choose which stock to buy or sell.

Result in app: Returns all Stock objects sorted alphabetically by ticker.

Q11 - Trade dialog price autofill

Location: src/com/tracker/dao/StockDAO.java:51

Java method: StockDAO.getLatestClosePrice(stockId)

Triggered by: TransactionDialog stock combo listener at lines 88-91 and initial autofill at lines 122-125

Query type: SELECT

Tables used: Market_Data

SQL:

```
SELECT close_price FROM Market_Data WHERE stock_id = ? ORDER BY date_id DESC LIMIT 1
```

Parameters: 1=selectedStock.getStockId()

Purpose: Autofills the trade price field with the newest seeded market close price.

Result in app: Returns a float latest close price, or 0.0 if not found.

Q12 - Dashboard recent transactions table

Location: src/com/tracker/dao/TransactionDAO.java:94

Java method: TransactionDAO.getRecentTransactions(portfolioId)

Triggered by: DashboardFrame.loadTransactions() at src/com/tracker/view/DashboardFrame.java:366

Query type: SELECT with JOIN

Tables used: Transaction, Stock

SQL:

```
SELECT t.*, s.ticker FROM Transaction t
JOIN Stock s ON t.stock_id = s.stock_id
WHERE t.portfolio_id = ?
ORDER BY t.timestamp DESC LIMIT 50
```

Parameters: 1=selectedPortfolio.getPortfolioId()

Purpose: Shows the latest trade activity for the selected portfolio.

Result in app: Returns recent Transaction objects with display ticker.

Q13 - BUY and SELL audit trail

Location: src/com/tracker/dao/TransactionDAO.java:130

Java method: TransactionDAO.logTransaction(portfolioId, stockId, type, quantity, price)

Triggered by: TransactionDAO.executeBuy() at line 34 and executeSell() at line 71

Query type: INSERT

Tables used: Transaction

SQL:

```
INSERT INTO Transaction (portfolio_id, stock_id, type, quantity, price, timestamp)
VALUES (?, ?, ?, ?, ?, NOW())
```

Parameters: 1=portfolioId, 2=stockId, 3='BUY' or 'SELL', 4=quantity, 5=price

Purpose: Records every executed trade before holdings are updated.

Result in app: Creates one transaction history row.

Q14 - Built-in dashboard market price simulator

Location: src/com/tracker/dao/StockDAO.java:78

Java method: StockDAO.simulateLatestPriceMovement()

Triggered by: DashboardFrame.startAutoRefresh() calls it every 10 seconds before refreshDashboard()

Query type: UPDATE with JOIN

Tables used: Market_Data

SQL:

```
UPDATE Market_Data md
JOIN (
  SELECT stock_id, MAX(date_id) AS latest_date
  FROM Market_Data
  GROUP BY stock_id
) latest
ON md.stock_id = latest.stock_id
AND md.date_id = latest.latest_date
SET
  md.open_price = md.close_price,
  md.close_price = ROUND(GREATEST(1, md.close_price * (1 + ((RAND() - 0.5) / 50))), 2),
  md.high = GREATEST(md.high, md.close_price),
  md.low = LEAST(md.low, md.close_price),
  md.volume = GREATEST(1000, ROUND(md.volume * (1 + ((RAND() - 0.5) / 10))));
```

Parameters: No user parameters. Uses RAND() inside MySQL to generate small random price/volume movement.

Purpose: Simulates market movement inside the Java app, so no second terminal or external market API is needed.

Result in app: Updates the latest Market_Data row for every stock; DashboardFrame then reloads holdings and recalculates current value and profit/loss.

Important Observations

- The project correctly uses PreparedStatement for all runtime SQL with user input. This is safer than string concatenation and helps prevent SQL injection.
- The DAO pattern keeps SQL in one layer. UI classes call methods like authenticate(), getHoldingsByPortfolio(), and executeBuy() instead of embedding SQL inside button handlers.
- The market simulator now runs inside the Java app through StockDAO.simulateLatestPriceMovement(), called by DashboardFrame every 10 seconds.
- BUY and SELL are multi-step operations. A BUY can run Q13 + Q6 + Q7 or Q13 + Q6 + Q8. A SELL can run Q6 + Q13 + Q8 or Q6 + Q13 + Q9.
- Trade execution is not wrapped in a database transaction. If logTransaction succeeds but holding update fails, the data can become inconsistent. For a stronger implementation, executeBuy and executeSell should use conn.setAutoCommit(false), commit, and rollback.
- Money values are stored as FLOAT in schema.sql. For real financial systems, DECIMAL is safer because it avoids floating-point rounding issues.
- Holding should ideally have UNIQUE(portfolio_id, stock_id). The current Java logic tries to prevent duplicates through findHolding(), but a database constraint would be stronger.